PromoGuard

Complete Implementation Plan

Security-first Windows desktop application that discovers legitimate free crypto reward campaigns, rejects anything that costs money, connects read-only to TON and Trust Wallet, and records everything in a local audit trail.

Document version	1.0, October 25, 2026
Project version	0.4.0 (Phases 0 to 4 delivered)
Overall completion	About 48% (60% code written, 36% verified working)
Platform	Windows 11, Electron, HTML/CSS/JavaScript, SQLite
Purpose	Single source of truth: what exists, what is missing, and the exact order to finish it

Important: this plan does not promise income. Rewards are controlled by each project. PromoGuard can only find, check and track opportunities. It never spends your money, signs transactions on its own, or asks for a seed phrase.

Contents

1. How to read this plan 4

1.1 Status labels used everywhere . . . 4

1.2 How the completion percentage is calculated . . . 4

1.3 Rules that never change . . . 4

2. Current completion (exact status) 5

2.1 Phase scorecard . . . 5

2.2 Evidence on hand . . . 5

2.3 Bugs already found and fixed . . . 5

3. Product goals and boundaries 6

3.1 What PromoGuard does . . . 6

3.2 What it will never do . . . 6

3.3 Social platforms and the web at large . . . 6

4. Architecture 7

4.1 Layers (bottom to top) . . . 7

4.2 Data flow . . . 7

4.3 Project folder structure (current) . . . 7

4.4 Planned additions . . . 7

5. Foundation (Phases 0 and 1) - step by step 8

5.1 Known foundation risks . . . 8

6. Backend: main process and security 9

6.1 Main process responsibilities . . . 9

6.2 Bridge functions (window.pg), grouped . . . 9

6.3 Secrets and credentials . . . 9

7. Database - every table and migration 10

7.1 Migration v1 (Phase 1) . . . 10

7.2 Migration v2 (Phase 2) . . . 10

7.3 Migration v3 (Phase 3 and 4) . . . 10

7.4 Planned migrations . . . 10

7.5 Status model (12 statuses) . . . 10

7.6 Backup, export and deletion . . . 11

8. Program logic: policy, discovery, review 12

8.1 Zero-payment policy (src/policy.js) . . . 12

8.2 Discovery engine (src/discovery.js, main.js) . . . 12

8.3 Review queue . . . 12

8.4 Planned discovery improvements (Phase 2b) . . . 12

9. Wallet connections - TON and Trust Wallet 14

9.1 TON (TON Connect), read-only . . . 14

9.2 Trust Wallet (WalletConnect v2), read-only . . . 14

9.3 Feature compatibility matrix . . . 14

9.4 Wallet security rules (enforced) . . . 15

10. Phase 5 - Free reward claim preparation (NOT BUILT) 16

10.1 Seven-stage claim pipeline . . . 16
10.2 Fee-free verification (the hardest part) . . . 16
10.3 Planned code and tests . . . 16
10.4 Monitoring receipts . . . 16
11. Phase 6 - Local AI and MCP (NOT BUILT) 17
11.1 Ollama . . . 17
11.2 MCP tools (read-only first) . . . 17
12. Phase 7 - Dashboard, notifications, health 18
12.1 Dashboard totals (always separate) . . . 18
13. Security model and threat register 19
14. Testing plan 20
14.1 Tests still to write . . . 20
14.2 Rules for reporting . . . 20
15. Packaging and Windows release 21
16. Services, credentials and costs 22
17. Master build order (what to do, in sequence) 23
17.1 Effort sizing (relative) . . . 23
18. Risks, limits and honest expectations 24
19. Final acceptance criteria (100% definition) 25
19.1 Percent complete by acceptance item . . . 25
20. Immediate next actions 26
20.1 For you (in order) . . . 26
20.2 For the build (after your results) . . . 26
20.3 Commands reference . . . 26
20.4 Documents in the project . . . 26
20.5 Glossary . . . 26

1. How to read this plan

This document has three jobs: (1) state exactly what is finished and how sure we are, (2) describe every part of the final application in depth, and (3) list the build steps in order with entry and exit criteria so each phase can be checked before the next begins.

1.1 Status labels used everywhere

Label	Meaning
IMPLEMENTED	Code exists in the project.
TESTED (AUTO)	Automated tests ran and passed. Wallet and network tests use mocks.
TESTED (RUNTIME)	The real Electron app was launched (Linux, virtual display) and the behavior observed.
NOT VERIFIED	Written but never run for real: Windows, live network, real wallet, installer.
NOT BUILT	Does not exist yet.

1.2 How the completion percentage is calculated

- Eight phases (0 to 7) from the master prompt, weighted equally.
- Each phase gets two scores: Code written, and Verified (tested including runtime or real-world checks).
- Overall = average of the two. Code that is written but unverified earns half credit.
- A previous message quoted 62%. That figure was wrong; the same method used earlier gives 48%. This plan uses 48%.

1.3 Rules that never change

- No deposits, purchases, staking, lending, borrowing, swaps, gambling, paid minting.
- No gas or any fee paid to claim: a campaign that needs gas is REJECTED even if the reward is free.
- No seed phrase, recovery phrase, private key or wallet password is ever requested, stored or logged.
- No automatic signing. Every wallet authorization needs your explicit approval inside the wallet.
- The zero-payment filter is enforced in program code. No setting and no AI tool can disable it.
- Estimated or pending rewards are never counted as received. Only on-chain confirmed receipts count.

2. Current completion (exact status)



2.1 Phase scorecard

Phase	Scope	Code	Verified	Overall	State
0	Inspect project, baseline audit	100%	90%	95%	Done
1	Foundation: install, tests, launch, database, installer	85%	55%	70%	Windows and installer pending
2	Discovery and verification engine	90%	55%	73%	Live feeds, scheduler unverified
3	Read-only TON wallet	85%	35%	60%	Needs manifest and testnet wallet
4	Read-only Trust Wallet (WalletConnect)	80%	30%	55%	Needs project ID and real pairing
5	Free reward claim preparation	0%	0%	0%	Not built
6	Local AI (Ollama) and MCP read-only tools	0%	0%	0%	Not built
7	Full dashboard, notifications, health	40%	20%	30%	Partial
	Average	60%	36%	48%	

2.2 Evidence on hand

Check	Result
Automated tests	53 of 53 passed (policy 8, discovery 4, database 4, phase 2 14, TON 12, EVM 11)
Runtime smoke test (Linux, virtual display, fresh data folder)	Passed: secure bridge present, no Node access in the page, styling applied, campaign add and duplicate detection, review start, bad feed/manifest/project ID rejected, refresh refused without connection, no signing functions exposed, inline script blocked by content security policy
Production dependency audit	0 vulnerabilities (build tools report findings; to review)
Windows 11 launch	NOT RUN
Windows installer build and packaged app	NOT RUN
Live RSS/Atom feeds	NOT RUN
Real TON or Trust Wallet connection	NOT RUN
Live TON Center and RPC responses	NOT RUN

2.3 Bugs already found and fixed

- Content security policy was header-only and did not apply to local pages: moved into the page.
- SQLite native module built for Node crashed inside Electron: rebuild step added, tests run under Electron Node.
- Rate limiter delayed the first request: fixed.
- Duplicate, half-finished Phase 4 code was already committed with node_modules: replaced by one tested version, node_modules untracked.
- A lingering timer could hang the test run: fixed.

3. Product goals and boundaries

3.1 What PromoGuard does

- Finds candidate campaigns from public sources you approve (RSS, Atom, GDELT news index, official project pages).
- Checks each one against your zero-payment rules and a scam-risk checklist.
- Keeps every unverified lead out of the claimable list until you confirm the official terms.
- Connects read-only to TON (TON Connect) and Trust Wallet (WalletConnect v2) so it can show addresses, balances and incoming transfers.
- Tracks claims and receipts with proof (transaction hash, network, time) and a full audit trail.

3.2 What it will never do

- Scrape logged-in social media (Facebook, X, Instagram, Snapchat) or bypass access controls, CAPTCHAs or rate limits.
- Guarantee or promise income or any specific reward.
- Mark a campaign eligible because of keywords, an amount, or an AI summary.
- Execute scripts or links taken from a campaign page.
- Pay a fee to claim, even a small one.

3.3 Social platforms and the web at large

Google, X and Facebook cannot be searched automatically in bulk. Scraping breaks their terms, logged-in content needs your account, and official APIs need keys and approval (X is paid). Plan: use public feeds and official project pages now; add official search or social APIs only after their current terms, pricing and quotas are documented and you approve (Phase 2b).

4. Architecture

4.1 Layers (bottom to top)

Layer	Technology	Responsibility	Trust level
Operating system	Windows 11	Credential store, notifications, file system	Trusted
Main process	Electron main (Node.js)	All network access, database, wallet SDKs, consent dialogs, scheduler	Trusted code
Secure storage	Electron safeStorage	Encrypts wallet session data and project ID. Memory-only if unavailable	Trusted
Database	SQLite (better-sqlite3)	Campaigns, history, wallet metadata, consent, audit	Local only
Bridge	preload.js, contextBridge	Fixed list of named functions. No generic IPC	Narrow gate
Interface	HTML, CSS, JavaScript renderer	Display and user input only. No network (CSP connect-src none)	Untrusted
External services	Feeds, TON Connect bridge, TON Center, WalletConnect relay, RPC servers	Third-party data and connections	Untrusted

4.2 Data flow

```
Public feed -> main process fetch (https, size cap, redirect checks)
  -> parse -> keyword filter -> dedupe -> policy check -> DISCOVERED
  -> UNVERIFIED (never eligible automatically)
  -> you start review -> you supply terms URL + confirm terms
  -> policy re-check -> ELIGIBLE_FOR_REVIEW or INELIGIBLE
  -> (Phase 5) claim prep -> your wallet approval -> confirmation on chain
  -> REWARD_RECEIVED only after chain evidence
```

4.3 Project folder structure (current)

```
promoguard/
  main.js              window, security, IPC handlers, scheduler
  preload.js           narrow bridge exposed as window.pg
  package.json, package-lock.json, .gitignore
  src/
    policy.js          zero-payment rules, URL checks
    status.js          12 statuses + allowed transitions
    store.js           campaign add/review/expiry logic
    discovery.js       feed fetch, parse, dedupe, limiter
    db.js              SQLite + migrations v1..v3
    ton/  networks.js provider.js connector.js store.js secure-storage.js
    evm/  networks.js rpc.js walletconnect.js
  renderer/            index.html style.css app.js
  test/                policy discovery db phase2 ton evm + run.js
  docs/                tonconnect-manifest.template.json
  README, PROJECT_STATUS, SECURITY_REVIEW, TEST_REPORT,
  API_AND_DATA_SOURCES, CHANGELOG
```

4.4 Planned additions

```
src/claims/   prepare.js  verify-fee-free.js  monitor.js    (Phase 5)
src/ai/       ollama.js  prompts.js  mcp-tools.js             (Phase 6)
src/health.js app health and sync status                      (Phase 7)
IMPLEMENTATION_PLAN.md  (this document, kept in the repo)
```

5. Foundation (Phases 0 and 1) - step by step

Goal: a reproducible, tested base that starts reliably on Windows 11. Nothing else is built on top until this passes.

Step	Action	Exit check	Status
1.1	Install Node.js LTS and Git on Windows 11. Record versions.	node -v and npm -v printed	Your PC
1.2	Unzip project, run npm install (also rebuilds SQLite for Electron).	No errors; node_modules created	NOT VERIFIED on Windows
1.3	Run npm test.	All tests pass	Done on Linux (53/53)
1.4	Run npm start. Add a test campaign.	Window opens, styled, campaign appears and survives restart	Done on Linux virtual display
1.5	Check database file created in the app data folder; reopen app.	Data persists	Done (automated + runtime)
1.6	Confirm discovery consent prompt appears before any request.	Prompt lists hosts; Cancel sends nothing	Code done; NOT VERIFIED by hand
1.7	Run npm run dist to build the NSIS installer.	Installer file produced	NOT RUN
1.8	Install the packaged app on a clean profile and repeat 1.4.	Same behavior as development	NOT RUN
1.9	Review npm audit findings in build tools; update or document.	No unaddressed high issue in shipped code	Open
1.10	Tag a git checkpoint (v0.4.0-verified).	Tag exists	After 1.2 to 1.8 pass

5.1 Known foundation risks

- Native SQLite module must match Electron. The postinstall rebuild handles it; if install fails on Windows, install Visual Studio Build Tools and retry.
- Sandbox was disabled only for the Linux root test. Confirm it is enabled on Windows.
- No code signing yet: Windows SmartScreen will warn on the installer. A certificate costs money; optional.

6. Backend: main process and security

6.1 Main process responsibilities

- Creates the window with contextIsolation on, nodeIntegration off, sandbox on, navigation and new windows denied.
- Registers every IPC handler through one gate that rejects calls not coming from the app window.
- Validates every argument (type, length, range) before use. Parameterized SQL only.
- Performs all network calls. The page itself cannot reach the network.
- Asks for consent in a native dialog before contacting any new service. Consent is recorded.
- Runs the scheduler and TON polling only after scoped consent.

6.2 Bridge functions (window.pg), grouped

Group	Functions	Notes
Settings	getSettings, setMinReward	Minimum reward default 0.01
Campaigns	addCampaign, listCampaigns	Runs fixed policy
Review	reviewStart, reviewReject, reviewEvidence	Only route to eligible
Discovery	listFeeds, addFeed, runDiscovery, enableSchedule, disableSchedule	Feeds must be public https
TON	tonState, tonSetManifest, tonSetNetwork, tonWallets, tonConnect, tonDisconnect, tonEnableData, tonRevokeData, tonRefresh, tonDashboard, tonExplorer, tonWipeLocal	Read-only
Trust Wallet	wcState, wcSetProjectId, wcSetRpc, wcConnect, wcSelect, wcEnableData, wcRefresh, wcDashboard, wcDisconnect, wcExplorer, wcWipeLocal	Read-only
Misc	clearWalletMetadata, openLink	Link opens only approved https domains

There is deliberately no function for signing, sending, approving, or running arbitrary commands. A runtime test confirms no exposed name contains sign, send, transaction or approve.

6.3 Secrets and credentials

Secret	Where stored	Never stored in
TON Connect session data	safeStorage encrypted file (memory-only if unavailable)	SQLite, renderer, logs
WalletConnect session data	safeStorage encrypted file via custom storage adapter	SQLite, renderer, logs
WalletConnect project ID	safeStorage encrypted file; renderer cannot read it back	Source code, SQLite
Future API keys (TON Center, search APIs)	safeStorage now; Windows Credential Manager option	Source code, git
Seed phrase, private key, wallet password	Never requested and never stored	Anywhere

7. Database - every table and migration

SQLite file in the app data folder. Foreign keys on, WAL mode. Migrations are append-only and each runs inside a transaction: a failure rolls everything back. Old data is never dropped.

7.1 Migration v1 (Phase 1)

Table	Key columns
settings	key, value (JSON)
reward_campaigns	id, name, publisher, source_url, network, token_id, reward_amount, eligibility, end_date, status, reasons, evidence, last_checked
wallet_connections	id, provider, address, network, first_connected, last_used
wallet_transactions	id, address, network, tx_hash, status, provider, ts
wallet_audit_events	id, event, provider, network, outcome, ts, info

7.2 Migration v2 (Phase 2)

- reward_campaigns gains dedupe_key and review_note. Old lowercase statuses mapped to the new set; an old "eligible" becomes UNVERIFIED because it had no evidence.
- status_history(id, campaign_id FK, from_status, to_status, reason, ts) with cascade delete.
- discovery_jobs(id, trigger, started, finished, feeds, found, added, errors).
- Indexes on dedupe_key, status, campaign_id.

7.3 Migration v3 (Phase 3 and 4)

- wallet_transactions gains direction, amount_nano, counterparty, lt; unique index on (network, tx_hash).
- wallet_snapshots(id, connection_id FK, network, native_balance_nano, tokens_json, retrieved, provider, freshness).
- wallet_permissions(id, provider, operation, scope, approved, revoked): one row per consent, revocable.

7.4 Planned migrations

Version	Phase	Change
v4	5	claim_attempts(id, campaign_id FK, network, wallet, state, fee_check_result, fee_check_evidence, approved_at, tx_hash, confirmed_at, failure_reason); campaign_evidence(id, campaign_id FK, url, fetched_at, content_hash, note)
v5	6	ai_summaries(id, campaign_id FK, model, prompt_version, summary, created, flagged_uncertain); mcp_grants(tool, granted, revoked)
v6	7	notifications(id, kind, ref_id, shown_at, dismissed); app_health(key, value, updated)
Each	All	Test: upgrade from every earlier version with sample data; test an interrupted migration rolls back

7.5 Status model (12 statuses)

Status	Meaning	Allowed next
DISCOVERED	Found in a feed	UNVERIFIED, REJECTED, EXPIRED
UNVERIFIED	Terms not confirmed. Do not claim	UNDER_REVIEW, REJECTED, EXPIRED
UNDER_REVIEW	You are checking official terms	ELIGIBLE_FOR_REVIEW, INELIGIBLE, UNVERIFIED, REJECTED, EXPIRED
ELIGIBLE_FOR_REVIEW	Evidence URL plus confirmed terms passed policy	UNDER_REVIEW, REJECTED, EXPIRED (Phase 5 adds AWAITING_USER_APPROVAL)
INELIGIBLE / REJECTED / EXPIRED	Terminal	None
AWAITING_USER_APPROVAL, CLAIM_SUBMITTED, REWARD_PENDING, REWARD_RECEIVED, FAILED	Claim states	Unreachable until Phase 5

7.6 Backup, export and deletion

- Planned: automatic copy of the database before each migration, and a manual export (JSON) of campaigns and history without wallet secrets.
- Implemented: wipe local TON data, wipe Trust Wallet data, clear wallet metadata.
- Never stored: seed phrases, private keys, passwords, raw session secrets in SQLite.

8. Program logic: policy, discovery, review

8.1 Zero-payment policy (src/policy.js)

Every campaign carries ten yes/no fields. A field must be explicitly confirmed false. Missing means unknown, and unknown stays UNVERIFIED.

Field	Rejects when true
requiresDeposit	Deposit or investment needed
requiresGas	Gas or network fee paid by you
requiresPurchase	Buying a token, NFT or subscription
requiresStaking	Staking, locking, lending
requiresPaidMint	Paid minting or registration
requiresSendingFunds	Sending funds to anyone
requiresUnlimitedApproval	Unlimited or unnecessary token approval
requestsSeedPhrase	Asks for seed, key or recovery phrase
unclearSignature	Signature with unclear effect
unverifiedSite	Unverified website

- Text scan rejects phrases such as seed phrase, private key, double your, guaranteed profit, send X TON, validate your wallet, unlock fee.
- URL checks flag non-https, credentials in URL, IP-address hosts, punycode look-alikes, link shorteners, and domains not on your approved list.
- Reward below the minimum (default 0.01 units of the stated token) is rejected. Tokens are identified by network plus contract, never by symbol, and 0.01 of different tokens is never treated as equal value.
- Expired campaigns become EXPIRED.

8.2 Discovery engine (src/discovery.js, main.js)

Control	Value
Protocol	https only; localhost, private and IP hosts blocked
Redirects	Manual, maximum 3, each target re-validated
Timeout / size	15 seconds; 2 MB cap (streamed)
Retries	Up to 2 on timeout, 429 or 5xx, exponential delay; none on 4xx
Rate limit	At least 2 seconds between requests to one host
Formats	RSS, Atom, GDELT JSON; malformed feeds raise an error
Relevance	Keyword filter (airdrop, giveaway, learn and earn, reward, bounty, campaign, promo)
Dedupe	Key = host + path + query without tracking parameters
Consent	Manual run: dialog each time. Scheduled run: one consent bound to exact hosts and interval; changing either asks again
Output	Everything starts UNVERIFIED. Discovery can never create ELIGIBLE_FOR_REVIEW

8.3 Review queue

- You press Start review on an UNVERIFIED item.
- You enter the official terms URL (https), the published reward amount and token ID, and tick the box only if the terms confirm no payment of any kind.
- The policy runs again. Pass plus evidence URL gives ELIGIBLE_FOR_REVIEW. A payment flag gives INELIGIBLE. Anything missing keeps UNDER_REVIEW.
- Every step is written to status_history with a reason.

8.4 Planned discovery improvements (Phase 2b)

- Official project campaign pages and documented announcement feeds, added by you.
- Optional search and social APIs: document current terms, pricing, quotas first (Google Programmable Search, X API, public Telegram or Reddit readers). Add only with your approval and credentials stored in safeStorage.
- Source reputation score from your own approvals and rejections.
- Notification when a tracked campaign is near expiry.

9. Wallet connections - TON and Trust Wallet

9.1 TON (TON Connect), read-only

Item	Design	Status
SDK	@tonconnect/sdk 5.0.0, runs in main process	Implemented; loads in Electron
Manifest	Public https tonconnect-manifest.json (url, name, iconUrl). You host it; PromoGuard never publishes it	Template in docs/. Blocked on you
Networks	TON Mainnet (chain -239), TON Testnet (chain -3). Testnet is default; mainnet needs a warning confirmation	Implemented, tested (mock)
Connection flow	Pick wallet from SDK list, consent dialog, open wallet link, approve in wallet	NOT VERIFIED live
Mismatch	Wallet network different from selected network: warning, no data requested. Unknown chain: disconnect	Tested (mock)
Data provider	TON Center API v3 (balances, jetton wallets, transactions). Free use about 1 request per second without key	Implemented; endpoint fields NOT VERIFIED live
Amounts	Integer math only (nanoTON), no floating point	Tested
Tokens	Identified by network plus jetton master address; metadata labeled unverified	Tested
Incoming transfers	Source present, destination is you, value above 0. Status: confirmed on-chain (master block present), failed (aborted), unconfirmed	Tested (mock)
Consent	Address sent to TON Center only after a recorded consent per network, revocable	Tested
Not exposed	sendTransaction, signData, signMessage, ton_proof	Tested

9.2 Trust Wallet (WalletConnect v2), read-only

Item	Design	Status
SDK	@walletconnect/sign-client 2.26.0 plus @walletconnect/core with custom encrypted storage	Implemented; loads in Electron
Credential	Project ID from the Reown dashboard (free-tier terms not verified)	Blocked on you
Pairing	QR code from a pairing URI shown once, scanned in Trust Wallet. URI never stored or logged	Implemented; NOT VERIFIED live
Namespaces	Optional eip155 chains only, methods list empty, events chainChanged and accountsChanged	Tested (mock). Wallet acceptance of empty methods NOT VERIFIED
Networks	BNB Smart Chain 56 and testnet 97 with official public RPC. Ethereum 1 and Sepolia need a user-supplied RPC	Implemented
Chain safety	eth_chainId must match the registry before any balance read	Tested
Balance	eth_getBalance, 18-decimal integer formatting	Tested (mock)
Events	Account change, chain change, remote disconnect, restore on start, timeout, rejection	Tested (mock)
Not supported	Token balances, transaction history, incoming-transfer tracking (need an indexer service and key). Only an observed balance rise, labeled not a receipt	By design
Not exposed	request(), personal_sign, eth_sendTransaction, typed-data signing	Tested

9.3 Feature compatibility matrix

Feature	TON	Trust Wallet (BSC)	Trust Wallet (Ethereum)
Connect / disconnect	Yes	Yes	Yes
Public address and network	Yes	Yes	Yes
Native balance	Yes	Yes (preconfigured RPC)	Needs your RPC

Feature	TON	Trust Wallet (BSC)	Trust Wallet (Ethereum)
Token balances	Jettons (provider-dependent)	No	No
Incoming transfers	Yes (provider-dependent)	No (balance change only)	No
Sign or send	Never	Never	Never
Testnet	Yes	Yes (97)	Yes (Sepolia)

9.4 Wallet security rules (enforced)

- Never ask for, store or log seed phrases, private keys or passwords. No recovery or import screen exists.
- A connected wallet is not proof of identity or eligibility and is not permission to spend.
- Mainnet is always labeled as real assets; testnet balances are labeled no real value.
- Never open or run connection links taken from promotions without validation and your confirmation.
- Provide disconnect and a button to erase local wallet data.

10. Phase 5 - Free reward claim preparation (NOT BUILT)

Entry criteria: Phases 1 to 4 verified on Windows with a real testnet connection. Without that, Phase 5 does not start.

10.1 Seven-stage claim pipeline

Stage	What happens	Hard rule
1 Recheck	Fetch the official terms again; save a content hash as evidence	Terms changed means back to UNDER_REVIEW
2 Active	Confirm campaign dates and distribution status	Expired or closed means EXPIRED
3 Eligibility	Check published requirements against what is verifiable	Cannot verify means UNVERIFIED, do not claim
4 Destination	Verify supported chain and the receiving wallet and network match	Mismatch blocks the claim
5 Fee check	Determine if gas, payment, token approval or any commitment is needed	Any fee or unclear cost means REJECT
6 Explain	Show exact action, network, contract, token, amount, consequences	Undecodable data means warn and stop
7 Approve and monitor	Only a permitted wallet authorization, approved by you inside the wallet; then track the transaction hash to confirmation	Never auto-sign; never count pending as received

10.2 Fee-free verification (the hardest part)

- Gas is paid by the sender. A claim you send yourself always costs gas, so it fails the zero-payment rule.
- Allowed paths are therefore: direct distribution to your address (monitor and record the receipt, send nothing), or a sponsored or relayed claim where the project pays and this can be proven from documentation and a simulated transaction showing zero cost to you.
- If fee-free execution cannot be proven, the campaign stays rejected. There is no override.
- Token approvals: unlimited or unnecessary approvals are rejected outright.

10.3 Planned code and tests

- src/claims/verify-fee-free.js, prepare.js, monitor.js; migration v4; new IPC functions for approve-intent only (no signing in PromoGuard).
- Tests: payment-required claim rejected, hidden-fee terms, malicious approval request, fake receipt, unconfirmed counted as received (must fail), mainnet/testnet confusion, cancelled and rejected signatures, duplicate claim.

10.4 Monitoring receipts

- TON: poll TON Center for incoming transfers; match amount and token to the campaign; require confirmed status.
- EVM: needs an indexer API with a key. Until then rely on balance change plus the official campaign confirmation, and label it unconfirmed.
- REWARD_RECEIVED requires chain evidence: hash, network, time.

11. Phase 6 - Local AI and MCP (NOT BUILT)

11.1 Ollama

- Free, open-source local runtime. Model choice depends on your RAM; a small 3B to 8B instruct model is the starting point.
- Uses: summarize campaign rules, extract amounts and deadlines, compare terms with the zero-payment policy, explain risk indicators, write evidence summaries.
- AI output is advisory and labeled with uncertainty. It is never the sole verification and can never change a status by itself.
- Campaign text, token names and contract metadata are untrusted input: strip markup, cap length, wrap in a data fence, and ignore instructions inside it (prompt-injection defense).
- The zero-payment setting is not reachable by any AI tool.

11.2 MCP tools (read-only first)

Tool	Returns	Permission
get_connected_wallets	Provider and network only	Read
get_public_wallet_address	Public address, only after consent	Read
get_wallet_balance	Last stored snapshot with time and source	Read
get_supported_token_balances	Stored token list, unverified labels kept	Read
get_recent_transactions	Stored incoming transfers	Read
get_network_status	Selected and wallet networks, mismatch	Read
analyze_reward_campaign	Evidence-based summary	Read
check_campaign_eligibility	Policy result and reasons	Read
explain_transaction_request	Decoded explanation or a warning it cannot be decoded	Read

- Strict input validation, narrow output, explicit consent per tool. No private keys, no filesystem or command access, no signing.
- Any future tool that can act externally needs its own separate authorization and never signs for you.

12. Phase 7 - Dashboard, notifications, health

12.1 Dashboard totals (always separate)

Total	Counts only
Rewards discovered	Campaigns found
Eligible for review	ELIGIBLE_FOR_REVIEW
Claims submitted	CLAIM_SUBMITTED
Pending confirmation	REWARD_PENDING
Confirmed received	REWARD_RECEIVED with on-chain evidence

- Screens: wallets and networks, balances, campaigns with evidence, eligible list, rejected list with reasons, pending claims, confirmed rewards, incoming transfers, discovery history, security alerts, application health and last successful sync.
- Notifications: new candidates, new confirmed incoming transfer, campaign near expiry, failed claim, disconnection. Each can be turned off.
- Currently built: campaign totals, status list with history, TON and Trust Wallet panels. Missing: alerts, health, receipts list, charts.

13. Security model and threat register

Threat	Control	Status
Phishing and look-alike domains	Approved-domain list, punycode and IP checks, shortener flag	Tested (auto)
Fake or fee-hiding campaigns	Unknown means unverified; evidence URL required; text scan	Tested (auto)
SSRF and local network access	https only, private hosts blocked, redirect re-validation	Tested (auto)
Malformed or huge feeds	Parser errors, 2 MB streamed cap, timeouts	Tested (auto)
SQL injection and bad input	Parameterized SQL, argument validation	Tested (auto)
Cross-site scripting	Text set via textContent, strict CSP, no inline script	Tested (runtime)
Renderer privilege escalation	Context isolation, sandbox, narrow bridge, sender check	Tested (runtime, partial)
Wrong network or token	Chain id checks, network plus contract identity	Tested (auto)
Fake receipts	Only chain-confirmed status counts	Tested (auto, TON)
Malicious approval requests	No methods requested; Phase 5 rejects unlimited approvals	Phase 5
Secret leakage	safeStorage, no secrets in SQLite or logs	Tested (auto)
Supply-chain risk	Lockfile, audit, review of new packages before install	Open
Prompt injection	Untrusted-data fencing, AI cannot change status	Phase 6

14. Testing plan

Suite	Tests now	Covers
policy	8	Zero-payment rules, URLs, minimum, expiry, token identity
discovery	4	Feed URL safety, RSS and GDELT parsing, relevance
db	4	Fresh migrate, persistence, injection safety, no secret columns
phase2	14	Atom, malformed feeds, dedupe, size cap, redirects, retries, rate limit, status machine, evidence gate, expiry, v1 upgrade, rollback
ton	12	Networks, formatting, parsing, provider errors, mocked connector, storage, consent, v3 upgrade
evm	11	CAIP-10, registry, RPC chain check, mocked WalletConnect, storage, consent
Total	53	All passing under Electron Node

14.1 Tests still to write

- Windows end-to-end: launch, add, review, restart, wipe.
- Live feed fetch against a real public feed.
- Testnet TON connection with a real wallet; testnet BSC connection with Trust Wallet.
- Claim pipeline suite (Phase 5) and AI/MCP injection suite (Phase 6).
- Installer install, upgrade-in-place and uninstall tests.
- Interrupted-migration test for each new version.

14.2 Rules for reporting

- A test is reported as passed only if it was executed and passed.
- Reports must say which kind of evidence: source review, syntax check, automated test, runtime test, or real-network check.

15. Packaging and Windows release

Step	Action
1	npm install, npm test, npm start on Windows 11
2	npm run dist (electron-builder, NSIS target) produces the installer
3	Install on a clean Windows profile; run the smoke checklist
4	Verify: sandbox on, CSP active, database created in app data, no seed or key fields anywhere
5	Optional: code-sign the installer (paid certificate) to avoid SmartScreen warnings
6	Optional: auto-update channel (needs a hosted release location and approval)
7	Tag release, write CHANGELOG, update PROJECT_STATUS

- Browser preview (promoguard-preview.html) is an old Phase 1 demo with in-memory data. It does not match the app and is not part of the release.

16. Services, credentials and costs

Service	Purpose	Credential	Cost / limit (verified?)
TON Connect bridge and wallet list	Wallet connection	None; needs your hosted manifest	Not documented here
TON Center API v3	Balances, tokens, transfers	Optional key	About 1 request per second without a key (checked); paid plans exist
WalletConnect / Reown relay	Trust Wallet session	Project ID (free signup; terms NOT verified)	Not verified
BNB Chain public RPC	BSC balance	None	About 10,000 requests per 5 minutes (secondary sources)
Ethereum / Sepolia RPC	Balance	Depends on provider you choose	Provider terms apply
GDELT, Google News RSS, project feeds	Discovery leads	None	Public; respect terms
Google Programmable Search / X API	Optional discovery	Keys required	Not verified; X is paid
Ollama	Local AI	None	Free, uses your computer
Hosting for manifest	TON Connect	Domain and HTTPS host	Free options exist; your choice
Code-signing certificate	Installer trust	Purchase	Paid, optional

17. Master build order (what to do, in sequence)

Do not skip ahead. Each phase has an entry check and an exit check. Work in small reviewable steps and stop for approval before higher-risk phases.

#	Phase and task	Entry	Exit	Owner
1	Windows verification of Phases 1 to 2: install, test, launch, installer, live feed	Zip delivered	Checklist in section 5 all passes; tag v0.4.0-verified	You run, Claude fixes
2	Host TON manifest; create throwaway testnet TON wallet	Step 1 done	Manifest URL reachable over https	You
3	Verify Phase 3: connect testnet wallet, allow lookups, refresh, view transfers, confirm endpoint fields	Step 2	Real balance and a testnet transfer shown with correct status; parser fixed if fields differ	You test, Claude fixes
4	Create Reown project ID; testnet BSC account in Trust Wallet	Step 1	Project ID saved in app	You
5	Verify Phase 4: QR pairing, balance, account/chain change, disconnect; resolve empty-methods question	Step 4	Real pairing works or documented workaround added	You test, Claude fixes
6	Phase 2b: official project pages and optional APIs (terms documented first)	Steps 1 to 5 stable	Sources documented in API_AND_DATA_SOURCES.md, tested	Claude with your approval
7	Phase 5: claim preparation with fee-free verification, migration v4, receipts monitoring	Steps 3 and 5 verified	Claim suite passes; payment-required claims always rejected	Claude, your approval first
8	Phase 6: Ollama summaries, read-only MCP tools, injection tests	Step 7	AI cannot change status or touch zero-payment setting	Claude
9	Phase 7: full dashboard, alerts, health, notifications	Steps 7 and 8	All five totals separate; alerts working	Claude
10	Hardening: dependency review, full security review, performance, documentation	Step 9	SECURITY_REVIEW and TEST_REPORT complete; no open high issue	Claude
11	Release: signed installer (optional), final Windows test, tag v1.0	Step 10	Acceptance criteria in section 19 met	You and Claude

17.1 Effort sizing (relative)

Block	Relative effort	Main risk
Verification steps 1 to 5	Small to medium (mostly your testing)	Real wallets behave differently from mocks
Phase 2b	Small	API terms and keys
Phase 5	Large	Proving zero cost; many campaigns will simply be rejected
Phase 6	Medium	Prompt injection, local model quality
Phase 7	Medium	Scope creep
Hardening and release	Medium	Packaging and signing

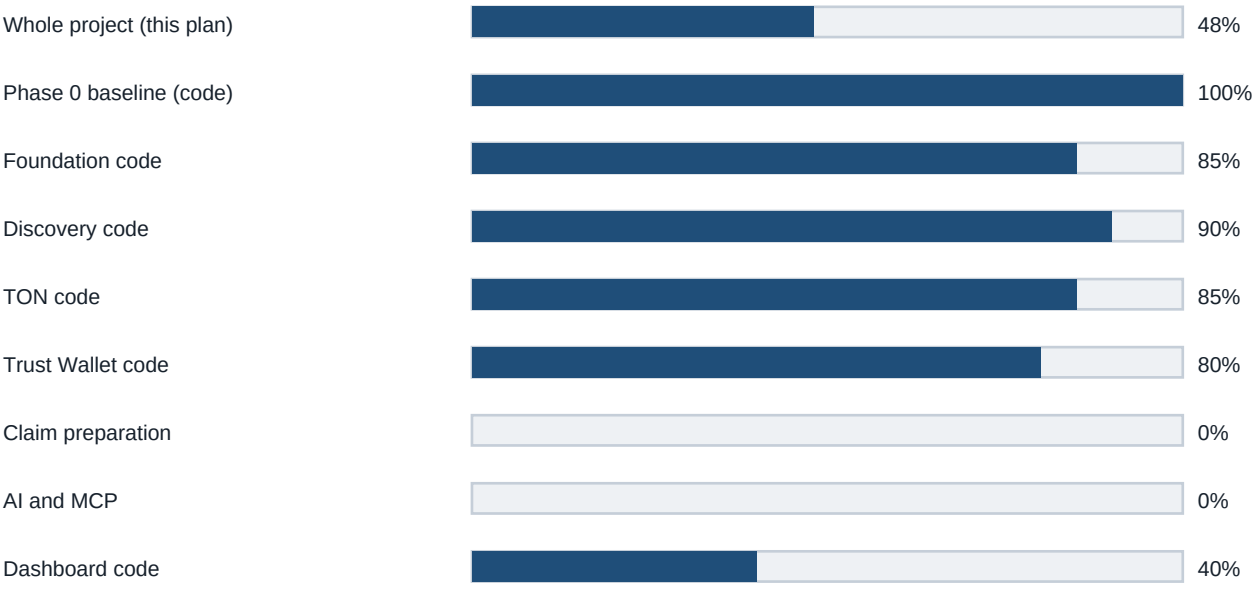
18. Risks, limits and honest expectations

- Free rewards are rare and controlled by each project. Many campaigns require gas or purchase and will be rejected. The app may show few eligible items. That is correct behavior.
- Gas is the key constraint: a self-sent claim always costs gas. Genuinely free paths are direct distributions and project-sponsored claims.
- Scammers copy official names. A mention of TON or Trust Wallet is never proof of legitimacy.
- Mocks do not equal real wallets. Phases 3 and 4 stay marked NOT VERIFIED until real testnet connections work.
- Open-source local AI models can be wrong. Their output never decides eligibility.
- Third-party free tiers and endpoints change. Every external field is parsed defensively and re-checked against current documentation before release.
- This plan is a software plan, not financial or legal advice.

19. Final acceptance criteria (100% definition)

#	Criterion	Done today?
1	Windows app launches and runs reliably	No (Linux only)
2	Database data preserved across upgrades	Yes (automated)
3	Discovery and zero-payment filtering pass automated and runtime tests	Automated yes; live no
4	Installer builds and the packaged app is tested	No
5	TON and Trust Wallet connections work as documented with real wallets	No
6	Balances and incoming transfers verified	No (TON mocked; EVM transfers unsupported)
7	Eligibility decisions carry traceable evidence	Yes
8	Claim workflow rejects every payment-required campaign	No (Phase 5)
9	Any wallet authorization needs explicit user approval in the wallet	Design yes; claim flow not built
10	Received rewards confirmed on-chain, never estimated	Partly (TON status model)
11	Security and privacy controls pass testing	Partly
12	Documentation matches the tested implementation	Yes, kept updated

19.1 Percent complete by acceptance item



20. Immediate next actions

20.1 For you (in order)

- Install Node.js LTS on Windows 11, unzip promoguard.zip, run npm install, npm test, npm start, then npm run dist. Send me the exact error text if anything fails.
- Add one public feed in the app and run discovery once. Tell me what happens.
- Host the TON Connect manifest (template in docs/) and make a throwaway TON testnet wallet.
- Create a free Reown project ID and a throwaway BSC testnet account in Trust Wallet.

20.2 For the build (after your results)

- Fix anything the Windows run exposes; tag the verified version.
- Correct the TON Center and RPC parsers using real responses.
- Begin Phase 2b, then Phase 5 only after steps 3 and 5 in section 17 pass.

20.3 Commands reference

```
npm install      install and rebuild SQLite for Electron
npm test         run all 53 tests
npm start        launch the app
npm run dist     build the Windows installer
PG_SMOKE=1 npm start   runtime smoke test hook (development only)
```

20.4 Documents in the project

File	Content
PROJECT_STATUS.md	Per-phase status and evidence
SECURITY_REVIEW.md	Security issue register
TEST_REPORT.md	Test evidence
API_AND_DATA_SOURCES.md	Services, limits, compatibility matrix
CHANGELOG.md	Version history
README.md	Setup
IMPLEMENTATION_PLAN.md	Copy of this plan (to be added to the repo)

20.5 Glossary

Term	Meaning
TON Connect	Standard way for apps to connect to TON wallets
WalletConnect / Reown	Standard way for apps to connect to many wallets, including Trust Wallet
Manifest	Public JSON file describing your app to TON wallets
RPC	Server that answers blockchain questions such as balance
Gas	Network fee paid by whoever sends a transaction
Jetton	Token standard on TON
CAIP-10	Account format such as eip155:56:0xabc...
safeStorage	Electron feature that encrypts data using the operating system
Testnet	Practice network whose coins have no real value
MCP	Protocol that lets an AI call tools; here read-only